Apuntes del Sprint 9 · Aprender sin respuestas: clustering, PCA, recomendadores y refuerzo

Las diapositivas del Sprint 9 cubren **tres tipos de aprendizaje distintos** en pocas páginas. Estos apuntes **explican la idea de cada uno con ejemplos pequeños que puedes calcular a mano**. Léelos antes de cada cuaderno.

0. De qué va el sprint (en 1 minuto)

Hasta ahora **siempre había una columna objetivo** (¿cancela? ¿cuánto cuesta?). En este sprint, **no**:

Presentación	Pregunta	Tipo de aprendizaje
S09.1 Clustering y PCA	¿Qué «tipos» de cliente hay? ¿Cómo dibujo datos de muchas columnas?	**No supervisado**
S09.2 Recomendadores	¿Qué experiencias le gustarán a cada huésped?	**No supervisado / híbrido**
S09.3 Refuerzo	¿Cómo aprende un agente probando, con premios y castigos?	**Por refuerzo**

El reto: segmentar a los huéspedes de TurisData, construir un recomendador de experiencias, ponerle **nombre, tamaño y acción comercial** a cada segmento y reflexionar sobre **IA responsable**.

> **Lo más importante del sprint:** el algoritmo da *grupos* o *listas*, pero **tu trabajo no termina ahí**: tienes que **darles significado de negocio**. Un grupo sin nombre no sirve a nadie.

DÍA 1 · Clustering y PCA

1. Aprender sin respuestas

Supervisado	No supervisado
Hay una columna objetivo (`cancelada`). El modelo aprende a **predecirla**	**No hay objetivo.** El algoritmo **descubre estructura**: grupos (*clustering*) o ejes de variación (PCA)

El caso: Lucía, directora comercial, envía **la misma newsletter a sus 1.200 huéspedes**. Sospecha que hay «tipos» de cliente muy distintos. **No hay una columna que diga el tipo**: hay que descubrirlo.

2. Escalar primero (otra vez)

El clustering **agrupa por distancia**. Si el gasto va en euros (cientos) y `viaja_con_ninos` es 0/1, **el gasto manda y el 0/1 no cuenta**.

Ejemplo: dos huéspedes con gasto 500 € y 300 €, y uno viaja con niños y otro no. La distancia = $\sqrt{(500-300)^2 + (1-0)^2} = \sqrt{40.000 + 1} \approx$ **200,0025**. Los niños **no aportan casi nada** a la distancia.

```
X = StandardScaler().fit_transform(huespedes.drop(columns="id_huesped"))      # 1200 × 7
```

> **Pregunta de reflexión del cuaderno:** prueba sin escalar y **compara la silueta**.

3. k-means

Idea: el algoritmo de **los centros que se mueven**.

1. **Elige k centros** (al azar al principio).
2. **Asigna cada huésped al centro más cercano.**
3. **Recalcula cada centro** (la media de sus huéspedes).
4. **Repite** hasta que nada cambie.

Ejemplo en una línea con 6 huéspedes cuyo gasto (en cientos de €) es `1, 2, 3, 10, 11, 12` y k = 2, con centros iniciales en 1 y 12:

Paso	Qué ocurre
Asignar	1, 2, 3 → centro 1 (el más cercano). 10, 11, 12 → centro 12
Recalcular	Centro A = (1+2+3)/3 = **2**; Centro B = (10+11+12)/3 = **11**
Repetir	Asignaciones iguales → **fin**

Resultado: dos grupos, **«gasto bajo» y «gasto alto»**.

Rasgos de k-means: rápido y fácil de explicar; **tú decides k**; prefiere grupos «redondos» y de tamaño parecido.

Cómo elegir k: el coeficiente de silueta

Mide **cómo de bien encaja cada punto en su grupo** frente al grupo vecino. Para cada punto:

```
a = distancia media a los de SU grupo
b = distancia media a los del grupo más cercano (el otro)
silueta = (b − a) / máx(a, b)
```

Ejemplo con el huésped de gasto 3: a = (|3−1| + |3−2|)/2 = **1,5**; b = (7 + 8 + 9)/3 = **8**. Silueta = (8 − 1,5)/8 = **0,81** (muy bien encajado).

Silueta media	Lectura
> 0,5	Estructura buena
0,25 – 0,5	Aceptable
≈ 0 o negativa	Los grupos se mezclan

```
siluetas = {k: silhouette_score(X, KMeans(k, n_init=10, random_state=SEMILLA).fit_predict(X)) for k in range(2, 9)}
mejor_k = max(siluetas, key=siluetas.get)
```

> **La silueta más alta no siempre da el mejor número de segmentos para el negocio.** Quizá k = 2 puntúa más, pero a Lucía le sirven más 4 segmentos que pueda atender con 4 newsletters distintas.

4. Interpretar: ¿quién es cada grupo?

```
perfil = huespedes.assign(grupo=etiquetas).groupby("grupo").mean()
grupo_familias = perfil["viaja_con_ninos"].idxmax()
grupo_aventura = perfil["pct_gasto_actividades"].idxmax()
```

Se calcula **la media de cada variable por grupo** y se lee como un **cliente típico**: edad, gasto, niños, % de gasto en actividades. Un **mapa de calor** de esas medias da la pista del nombre.

Ejemplo de ficha de segmento:

Grupo	Rasgos	Nombre	Acción comercial
0	70 % viaja con niños, gasto medio	**«Familias»**	Oferta de actividades infantiles y habitaciones familiares
1	28 años, 45 % del gasto en actividades	**«Aventura»**	Paquetes de senderismo y buceo
2	62 años, alto gasto en restaurante y spa	**«Descanso premium»**	Servicios de bienestar y estancias largas

(Cifras ilustrativas.) **Cada grupo = nombre + descripción en una línea + acción comercial.** Eso es lo que pide el mini-reto.

5. Jerárquico y DBSCAN

	Jerárquico (Ward)	DBSCAN
Idea	Cada huésped empieza **solo**; se **fusionan los más parecidos**, paso a paso	Los grupos son **zonas densas** de puntos

	Jerárquico (Ward)	DBSCAN
¿Hay que decir k?	Se «corta» el **dendrograma** a la altura que da k	**No hace falta decir cuántos**
Parámetros	`n_clusters`, `linkage="ward"`	**`eps`** (radio) y **`min_samples`**
Rarezas	—	Lo que no encaja en ninguna zona densa es **ruido (−1)**: clientes raros

El dendrograma es un árbol genealógico al revés: abajo, cada huésped; según subes, se van uniendo los más parecidos. Cortarlo con una línea horizontal da los grupos.

```
AgglomerativeClustering(n_clusters=4, linkage="ward")
DBSCAN(eps=1.0, min_samples=10)
```

DBSCAN, intuición: un punto está en una zona densa si tiene al menos `min_samples` vecinos a menos de `eps`. Los puntos aislados quedan como **ruido**: a veces son los clientes más interesantes (o errores).

> **Mini-reto:** elige método, **justifícalo en una frase** y escribe para cada grupo nombre, descripción y acción comercial (`nombres_segmentos`).

6. PCA: ver los datos en 2D

Con 7 variables no puedes dibujar los datos. **PCA** busca **nuevas «direcciones»** que resuman la variación:

> **Imagina una nube de puntos con forma de puro.** El **componente 1** es la dirección **más larga** (donde hay más variación); el **componente 2**, la siguiente, **perpendicular**.

Para qué sirve:

1. **Dibujar** (2 componentes → mapa 2D).
2. **Resumir**: quedarse con pocos componentes que conserven casi toda la información.

¿Cuántos componentes?

```
varianza_acum = np.cumsum(pca.explained_variance_ratio_)
n_comp_90 = int(np.argmax(varianza_acum >= 0.90)) + 1
```

Ejemplo: si cada componente explica el 55 %, 25 %, 12 %, 5 % y 3 %, la **varianza acumulada** es 55 %, 80 %, **92 %**, 97 %, 100 %. Para llegar al 90 % hacen falta **3 componentes**.

```
Z2 = PCA(n_components=2).fit_transform(X)        # mapa 2D
ax.scatter(Z2[:, 0], Z2[:, 1], c=grupo)          # coloreado por grupo
```

Pon en los ejes el % de varianza de cada componente («PC1 (55 %)») para que quien lo vea sepa cuánta información hay en el dibujo.

Leer un componente: es una receta

Un componente es **una combinación de variables**: `PC1 = 0,45·edad + 0,33·gasto − 0,44·%actividades …` Esos pesos se llaman **cargas** (*loadings*). **Variables con signo contrario** en un componente **se mueven en sentidos opuestos** (cuanto más edad, menos % de actividades). **El signo global es arbitrario**: puede venir al revés y no significa nada.

Usar PCA con cuidado

- **PCA antes de k-means** (`PCA(n_components=n_comp_90)`): a veces **limpia ruido**. Se comprueba la **estabilidad** de los grupos con `adjusted_rand_score` (1 = mismos grupos; ~0 = sin relación).
- **t-SNE / UMAP:** muy **bonitos** para ver grupos, pero **las distancias entre grupos no significan nada** y **cambian con la semilla**. Sirven para **mirar, no para medir**.

> **Mini-reto:** un compañero dice «con 2 componentes ya vale». *¿Para dibujar?* Sí. *¿Para un modelo que necesita precisión?* **Mira `varianza_acum`**: con el ejemplo de arriba, 2 componentes solo conservan el 80 %.

DÍA 2 · Sistemas de recomendación

7. Cómo probar un recomendador: esconder notas

TurisData vende **25 experiencias** (excursiones, spa, buceo…) con **los mismos banners para todos**. ¿Y si cada huésped viera **las que más le pueden gustar**?

Para saber si un recomendador es bueno, hay que esconderle algunas valoraciones y ver si las adivina.

- **Matriz usuario × experiencia** (300 usuarios, 25 experiencias): las casillas **vacías son `NaN`** (no han valorado esa experiencia).
- **Se reserva un 20 % al azar como prueba**; la matriz se construye **solo con el 80 %**.
- Dos formas de medir:

Métrica	Qué mide	Ejemplo
RMSE	Error en la **nota** predicha	Reales 4, 3, 5; predichas 3,5, 3, 4 → errores 0,5, 0, 1 → RMSE = √((0,25 + 0 + 1)/3) ≈ **0,65**
precision@k	De las **k recomendadas**, ¿qué fracción le gustó (≥ 4 estrellas en la prueba)?	Se recomiendan 3 y le gustan 2 → **2/3 = 0,67**

El usuario **no ve notas, ve una lista corta**: por eso **precision@k** se parece más a lo que importa. **Solo se recomienda entre lo no valorado** (no tiene sentido recomendar lo que ya hizo).

8. Tres enfoques

Enfoque	Idea	Pega
Popularidad	**Lo mejor valorado para todos**	**No personaliza.** Es la **referencia a batir** y la única opción en el **arranque en frío** (usuario nuevo)
Contenido	**Si te gusta el mar, más de mar** (perfil por categoría)	**Se queda en lo que ya conoces**: no te descubre nada
Colaborativo	**Usuarios (o experiencias) parecidos**, por similitud	Necesita **historial** y **falla con usuarios nuevos**

Analogía:

- **Popularidad:** el cartel «Lo más vendido».
- **Contenido:** el dependiente que te dice «como te gustó este libro de aventuras, mira esta saga».
- **Colaborativo:** «a las personas que compraron lo mismo que tú también les gustó…».

El perfil por contenido, en una línea

```
perfil_usuario = (C @ F) / ((M @ F) + 3)        # 300 × 7
```

- `F`: qué categoría tiene cada experiencia. `C`: **residuos** de cada usuario (nota − su media). `M`: qué ha valorado.
- Se **suma el residuo por categoría** y se divide entre **(nº de valoraciones + 3)**. El **«+3» es prudencia**: con pocas valoraciones no nos fiamos.

Ejemplo: un usuario valora 2 experiencias de mar con residuos +1 y +0,5 → perfil de «mar» = 1,5 / (2 + 3) = **0,30**. Si solo hubiera valorado 1 de mar (+1): 1/(1 + 3) = **0,25** (prudente). Con 10 valoraciones que suman 5: 5/13 = **0,38**.

9. Factorización de matrices: gustos ocultos

Idea: cada usuario tiene **unos pocos gustos ocultos** («aventura», «tranquilidad», «cultura») y cada experiencia **una intensidad en cada uno**. La nota esperada sale de **multiplicarlos**.

```
nota ≈ media + sesgo del usuario + sesgo de la experiencia + gustos · intensidades
```

Ejemplo:

- Media general = 3,5. Este usuario puntúa **más alto que la media** (+0,3). Esta experiencia puntúa **algo menos** (−0,2).
- Gustos del usuario: aventura = 1, tranquilidad = −0,5. Intensidades de la experiencia: aventura = 0,8, tranquilidad = 0,2.
- Gustos · intensidades = 1·0,8 + (−0,5)·0,2 = 0,8 − 0,1 = **0,7**.

- **Nota prevista = 3,5 + 0,3 − 0,2 + 0,7 = 4,3.**

```
P_mf = factorizar(R, k=3)                    # k = nº de gustos ocultos
rmse_mf, p_mf = evaluar_rmse(P_mf), precision_k(P_mf, 3)
```

Se **compara todo con RMSE** (error de la nota) **y con precision@3** (calidad de la lista). Puede que un método gane en una y pierda en otra.

10. Mini-reto: el recomendador de TurisData

```
def recomendar(id_usuario, n=3, P=P_mf):
    """n experiencias aún no valoradas con mayor nota prevista."""
```

Pasos: copia la fila del usuario de la matriz de predicciones → **pon −∞ donde ya valoró** (para no recomendarlo) → **ordena** y coge las n mejores. Prueba con los usuarios 1 y 2 y **comprueba que sus listas son distintas**: si no, estás recomendando lo popular.

> **IA responsable:** ¿qué pasa si **solo recomendamos lo popular**? Las experiencias poco conocidas **nunca se ven**, nunca se valoran y nunca suben: es un **bucle de retroalimentación** (Sprint 1). Un recomendador también **configura lo que la gente descubre**.

DÍA 3 · Aprendizaje por refuerzo

11. El bucle del refuerzo

Tercera gran familia: **un agente aprende probando, con recompensas y castigos** (como entrenar a un perro, o jugar a un videojuego sin manual).

Pieza	En el ejemplo del hotel
Agente	Quien decide: un empleado que quiere llegar a una habitación
Entorno	El pasillo del hotel: **estados** (casillas) y **acciones** (arriba, abajo, izquierda, derecha)
Recompensa	**+** si llega a la habitación con vistas; **−5** por pisar una zona en obras; **−** por cada paso (coste)

No hay respuestas correctas dadas: el agente aprende **qué acciones llevan a más recompensa total**.

El pasillo del hotel (cuaderno S09_04)

```
S . . . .        S = recepción (inicio)
. # # . X        # = pared
. . . . .        X = zona en obras (−5, se puede pisar)
. # . . G        G = habitación con vistas al mar (meta)
```

Es una cuadrícula de **4 × 5 = 20 estados**. La clase del entorno y la función `entrenar` ya están escritas; **tú programas `actualizar_q` y `elegir_accion`**.

12. La tabla Q y su regla de actualización

`Q[estado, acción]` = **la recompensa total esperada** si estoy en este estado, hago esta acción **y luego sigo lo mejor posible**. Es una tabla de 20 filas (estados) × 4 columnas (acciones). **Empieza en ceros**: el agente no sabe nada.

```
def actualizar_q(Q, s, a, r, s2, alpha, gamma):
    objetivo = r + gamma * Q[s2].max()          # lo que AHORA crees que vale la acción
    Q[s, a] += alpha * (objetivo - Q[s, a])     # mueve Q una fracción alpha hacia ese valor
```

Parámetro	Qué es
`alpha` (0–1)	**Cuánto corrige** cada experiencia
`gamma` (0–1)	**Cuánto pesa el futuro** frente al presente

Ejemplo numérico

El agente está en s, hace una acción a, recibe **r = −1** (coste del paso) y llega a s2, desde donde ya cree que lo mejor vale **Q[s2].max() = 4**. Con `alpha = 0,5` y `gamma = 0,9`:

```
objetivo = r + gamma · Q[s2].max() = −1 + 0,9·4 = −1 + 3,6 = 2,6

1.ª vez:  Q = 0     →  0 + 0,5·(2,6 − 0)     = 1,3
2.ª vez:  Q = 1,3   →  1,3 + 0,5·(2,6 − 1,3) = 1,95
3.ª vez:  se va acercando a 2,6
```

Cada experiencia **acerca la estimación al objetivo**, como afinar una opinión con cada dato nuevo. **La recompensa de la meta «se propaga hacia atrás»** por la tabla, casilla a casilla, hasta la recepción.

13. Explorar o explotar: epsilon-greedy

El dilema del restaurante: ¿vas **al de siempre** (que sabes que está bien) o **pruebas uno nuevo** (puede ser mejor… o peor)?

```
def elegir_accion(Q, s, epsilon, rng):
    if rng.random() < epsilon:
        return int(rng.integers(4))       # EXPLORAR: acción al azar
    return int(np.argmax(Q[s]))           # EXPLOTAR: la mejor que conozco
```

Con `epsilon = 0,1`: el **10 % de las veces explora** y el 90 % aprovecha lo aprendido.

- **Si siempre explota**, nunca descubre caminos mejores.
- **Si siempre experimenta**, nunca aprovecha lo aprendido.

14. Política aprendida y experimento

```
politica = Q.argmax(axis=1)        # la mejor acción en cada estado (20 acciones)
```

La **política** es el «mapa de instrucciones»: en cada casilla, qué hacer. El **camino** son los estados que visita el agente **sin explorar** desde recepción hasta la meta.

Experimento clave: con `coste_paso = 0`, **la única recompensa llega al final**; sin exploración, el agente **no encuentra nunca la meta y no aprende**. **La exploración importa.**

Mini-reto: pasar por la zona en obras cuesta ahora **−20** (reclamaciones). Reentrena y **comprueba si el camino cambia**: ahora el agente dará un rodeo, aunque sea más largo, porque el castigo supera el coste de unos pasos más.

---

15. Dónde practicar cada cosa

Si no entiendes…	Mira la sección	Practica en
Escalar, k-means, silueta, interpretar grupos	2-4	`S09_01_clustering`
Jerárquico y DBSCAN	5	`S09_01_clustering` (ejercicios 4 y 5)
PCA, varianza explicada, cargas	6	`S09_02_pca`
Recomendadores, RMSE, precision@k	7-10	`S09_03_recomendadores`
Q-learning, alpha, gamma, epsilon	11-14	`S09_04_q_learning`
Todo junto	todas	Reto: segmentos, mapa PCA, recomendador, ficha, IA responsable

16. Chuleta de vocabulario

Palabra	En lengua normal
No supervisado	Aprender sin columna objetivo
Clustering	Agrupar casos parecidos
k-means	Clustering por centros que se van moviendo

Palabra	En lengua normal
Silueta	Medida de lo bien que encaja cada punto en su grupo
Dendrograma	Árbol que muestra qué se fusiona con qué
DBSCAN	Clustering por zonas densas; lo demás es ruido
PCA	Resumir muchas variables en pocos componentes
Varianza explicada	Cuánta información conserva cada componente
Cargas	Pesos de cada variable en un componente
Arranque en frío	No hay historial de un usuario/producto nuevo
Filtrado colaborativo	Recomendar según gente parecida
Factorización de matrices	Descubrir «gustos ocultos»
precision@k	De las k recomendadas, cuántas gustan
Agente / entorno / recompensa	Quien decide / dónde actúa / premio o castigo
Tabla Q	Valor esperado de cada acción en cada estado
alpha / gamma / epsilon	Cuánto corrige / cuánto pesa el futuro / cuánto explora
Explorar / explotar	Probar cosas nuevas / aprovechar lo que ya sabes

17. Cómo estudiar estos apuntes

1. **Antes de cada cuaderno**, lee la sección indicada en la tabla del apartado 15.
2. **Calcula a mano** el ejemplo de la silueta (sección 3) y el de la **actualización de Q** (sección 12): con 3 números entenderás lo que hace el algoritmo.
3. **En el reto, no basta con el algoritmo:** cada segmento necesita **nombre, descripción y acción comercial**. Escribe como si se lo explicaras a Lucía.
4. **No olvides la reflexión de IA responsable:** ¿a quién deja fuera tu segmentación? ¿qué deja de ver quien solo recibe lo popular?